

Document Number: P1091R0

Date: 2018-05-07

Reply-to: Nicolas Lesser <blitzrakete@gmail.com>

Audience: EWG

# Extending structured bindings to be more like variable declarations

## Introduction

There are a lot of restriction on structured bindings compared to variable declarations, like not being able to mark them `static`, `constexpr` or them not having unclear linkage. This proposal's aim is to fix this by for example making the underlying structured binding object (and tuple binding variables) have external linkage and by allowing various specifiers (`static`, `thread_local`, `constexpr`, `inline`, `extern`) on structured bindings.

## Motivation

Structured bindings, although very useful, are actually pretty magical. They don't introduce variables in the normal sense for each binding, rather, they are names that refer to specific objects. As such, there are problems including what it means for a binding to be `static` and how it would work and what linkage do those bindings even have.

Those problems could not be resolved during the discussion of the paper and afterwards, a paper was requested to analyse the possible design impact that such additions to structured bindings would have after two NB comments proposing this were rejected. This paper attempts to do so.

One motivation to do so is to bring structured bindings closer to actual variable declarations, so consistency. This will also make structured bindings more useful, as they are currently lacking for example `constexpr`, which is becoming every more important for various features of the language.

As a consequence this paper also fixes some DRs that were filed and are under consideration or going to be eventually by either Evolution or Core.

## Linkage

As per [basic.link]p8, bindings do not have any linkage because they're just names. Currently, in both gcc and clang, tuple binding have linkage, as they were specified as variable declarations in the standard. This is no longer the case though due to the resolution of DR2313<sup>1</sup>.

But the underlying structured binding object is an actual variable, which can have either internal or external linkage depending on the declaration of the structured binding, as gcc and clang do it. There is no way to refer to that object though without making the program IL-NDR as the object has a name like `_ZDC1a2bbE` for `auto[a, bb]`.

[dcl.struct.bind]p1 has the following to say about the object:

First, a variable with a unique name *e* is introduced.

It follows that the variable cannot be referenced in a conforming program anyways, and as such, it doesn't make much sense to give it external linkage. Nonetheless, to be consistent with the rest of the declarations and being

able to use just `inline` (see below) without an extra `extern` to give the structured binding external linkage, the underlying object should have external linkage.

## **extern**

As discussed in the previous section, there is no way to reference either the bindings or the underlying object within a conforming program, so allowing `extern` on such a structured binding would not make much sense.

However, this would prohibit `inline` on structured bindings that have been declared `const` and `inline`, which might be desirable in some cases. For this reason, it should be allowed. Note that `extern` would have no effect on the individual bindings, except for the tuple case.

## **static and thread\_local**

`static` and `thread_local` on a structured binding make sense and are actually useful. The way to make this work nicely in the standard is to only apply them on the underlying object, and not on the bindings (which wouldn't make sense and can't work today without major changes to the specification of bindings anyways). For the tuple case on the additional variable declarations too.

Because bindings refer to certain objects (depending on the initializer of the structured binding), they would implicitly get the desired semantics of `static` and `thread_local`, as they refer to either objects within the underlying object or to separate variables (in the tuple case) which are marked with the desired specifiers.

## **inline**

`inline` is also useful and will be consistent with inline variables. It will work just as with `static` and `thread_local`: The underlying object is marked `inline` and any additional variables introduced as part of tuple bindings.

## **constexpr**

If `constexpr` were applied just like `static` and `co_` are, then there would be a problem, because the current language rules make the following code ill-formed:

```
// at block scope
constexpr auto[a] = std::tuple<int>(1);
// "equivalent" to
constexpr auto __sb = std::tuple<int>(1);
constexpr const int& __a = std::get<0>(__sb); // ill-formed today
```

A reference must be initialized by a constant expression to be a core constant expression ([`expr.const`]p2.11), but `std::get<0>(__sb)` is not a constant expression due to [`expr.const`]p6.

Richard Smith on the core reflector<sup>2</sup> suggested to relax the restriction on what constitutes a core constant expression of a reference by just requiring that the reference must be initialized by a *core* constant expression instead (see below to what this change entails).

This would mean that to make structured bindings `constexpr`, it is necessary to apply `constexpr` to the underlying object, and apply `const` to any other variable introduced by tuple bindings.

Of course, one thing to note is that it is important to guarantee that the call to `get` is a constant expression, because or else `constexpr` will act like `const`, which is only maybe a core constant expression.

## **Impact**

This proposal only makes ill-formed or code with unspecified behavior well-formed in relation to structured bindings.

Due to `constexpr` tuple binding variables requiring a change to what constitutes a core constant expression, ill-formed code today will become well-formed:

```
// at block scope
const int var = 1;
const int& ref = var;
static_assert(ref == 1); // ill-formed today, well-formed with this proposal
```

## Proposed wording

All changes relative to the latest C++20 draft.

Change [dcl.dcl]p9 as follows:

A *simple-declaration* with an *identifier-list* is called a *structured binding declaration* ([dcl.struct.bind]). The *decl-specifier-seq* shall contain only the *type-specifier* `auto`, the *storage-class-specifiers* `static`, `extern` and `thread_local`, `constexpr`, `inline` and *cv-qualifiers*. The initializer shall be of the form “= *assignment-expression*”, of the form “{ *assignment-expression* }”, or of the form “( *assignment-expression* )”, where the *assignment-expression* is of array or non-union class type.

Change [dcl.struct.bind]p1 as follows:

A structured binding declaration introduces the *identifiers*  $v_0, v_1, v_2, \dots$  of the *identifier-list* as names ([basic.scope.declarative]) of structured bindings. Let *cv* denote the *cv-qualifiers* in the *decl-specifier-seq*. First, a variable with a unique name *e* is introduced. For linkage purposes, two structured bindings with same  $v_i$  in the same order shall have the same name ([basic.link]). If the *assignment-expression* in the *initializer* has array type *A* and no *ref-qualifier* is present, *e* has type *cv A* and with every *decl-specifier* in the *decl-specifier-seq* applied except for the *type-specifier* `auto`, and each element is copy-initialized or direct-initialized from the corresponding element of the *assignment-expression* as specified by the form of the *initializer*. Otherwise, *e* is defined as-if by

attribute-specifier-seq<sub>[opt]</sub> decl-specifier-seq ref-qualifier<sub>[opt]</sub> *e* initializer ;

where the declaration is never interpreted as a function declaration and the parts of the declaration other than the *declarator-id* are taken from the corresponding structured binding declaration. The type of the *id-expression* *e* is called *E*. [ Note: *E* is never a reference type ([expr.prop]). The linkage of *e* is described in [basic.link] — end note ]

Change [dcl.struct.bind]p3 as follows:

Otherwise, if the *qualified-id* `std::tuple_size<E>::value` names a complete type, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the number of elements in the *identifier-list* shall be equal to the value of that expression. The *unqualified-id* `get` is looked up in the scope of *E* by class member access lookup ([basic.lookup.classref]), and if that finds at least one declaration that is a function template whose first template parameter is a non-type parameter, the initializer is `e.get<i>()`. Otherwise, the initializer is `get<i>(e)`, where `get` is looked up in the associated namespaces. In either case, `get<i>` is interpreted as a *template-id* and get shall be marked *constexpr* if *constexpr* is in the *decl-specifier-seq*. [ Note: Ordinary unqualified lookup is not performed. — end note ] In either case, *e* is an lvalue if the type of the entity *e* is an lvalue reference and an xvalue otherwise. Given the type  $T_i$  designated by `std::tuple_element<i, E>::type`, variables

are introduced with unique names  $r_i$  of type “reference to  $\underline{c} T_i$ ”, where  $c$  is *const* if *constexpr* appear in the *decl-specifier-seq* and empty otherwise, initialized with the initializer ([dcl.init.ref]), where the reference is an lvalue reference if the initializer is an lvalue and an rvalue reference otherwise, with every *decl-specifier* in the *decl-specifier-seq* applied except for the *type-specifier* *auto* and *constexpr*. If *constexpr* appears within the *decl-specifier-seq*, the initializer shall be a *constant-expression* and every  $T_i$  shall be of literal type. Each  $v_i$  is the name of an lvalue of type  $T_i$  that refers to the object bound to  $r_i$ ; the referenced type is  $T_i$ .

Change [expr.const]p2 as follows:

- an *id-expression* that refers to a variable or data member of reference type unless the reference has a preceding initialization and either
  - it is initialized with a core constant expression or
  - its lifetime began within the evaluation of  $e$ ;

Change [dcl.stc]p3 as follows:

The *thread\_local* specifier indicates that the named entity has thread storage duration. It shall be applied only to the names of variables of namespace or block scope, to structured binding declarations ([dcl.struct.bind]) and to the names of static data members. When *thread\_local* is applied to a variable of block scope the *storage-class-specifier* *static* is implied if no other *storage-class-specifier* appears in the *decl-specifier-seq*.

Change [dcl.stc]p4 as follows:

The *static* specifier can be applied only to names of variables and functions, to structured binding declarations ([dcl.struct.bind]) and to anonymous unions. There can be no *static* function declarations within a block, nor any *static* function parameters. A *static* specifier used in the declaration of a variable declares the variable to have static storage duration, unless accompanied by the *thread\_local* specifier, which declares the variable to have thread storage duration. A *static* specifier can be used in declarations of class members; [class.static] describes its effect. For the linkage of a name declared with a *static* specifier, see [basic.link].

Change [dcl.stc]p5 as follows:

The *extern* specifier can be applied only to the names of variables and functions and to structured binding declarations ([dcl.struct.bind]). The *extern* specifier cannot be used in the declaration of class members or function parameters. For the linkage of a name declared with an *extern* specifier, see [basic.link]. [ Note: The *extern* keyword can also be used in *explicit-instantiations* and *linkage-specifications*, but it is not a *storage-class-specifier* in such contexts. — end note ]

Change [dcl.inline]p1 as follows:

The *inline* specifier can be applied only to the declaration or definition of a variable or function, or to a structured binding declaration ([dcl.struct.bind]).

Change [dcl constexpr]p1 as follows:

The *constexpr* specifier shall be applied only to the definition of a variable or variable template or the declaration of a function or function template, or to a structured binding declaration ([dcl.struct.bind]). A function or static data member declared with the *constexpr* specifier is implicitly an inline function or variable ([dcl.inline]). If any declaration of a function or function template has a *constexpr* specifier, then all its declarations shall contain the *constexpr* specifier. [ Note: An explicit specialization can differ from the template declaration with respect to the

*constexpr* specifier. — end note ] [ Note: Function parameters cannot be declared *constexpr*. — end note ]

## References

<sup>1</sup>: [http://www.open-std.org/jtc1/sc22/wg21/docs/cwg\\_defects.html#2313](http://www.open-std.org/jtc1/sc22/wg21/docs/cwg_defects.html#2313)

<sup>2</sup>: <http://lists.isocpp.org/core/2018/04/4206.php>

## Acknowledgements

Thanks to k-ballo, T.C, Richard Smith, Alberto Barbati for mentioning and solving specific problems with this proposal and the countless others on the #future-standard Cpplang channel and the std-proposals mailing list.